

BÁO CÁO ĐỒ ÁN 1

Tìm hiểu Multicore

Nguyễn Đình Huy - 23520624

Nguyễn Gia Huy - 23520629

April 11th, 2026

Abstract

Báo cáo trình bày kết quả tìm hiểu về kiến trúc vi xử lý đa nhân (Multicore). Nội dung tập trung phân tích cấu trúc phần cứng, cơ chế quản lý tài nguyên dùng chung và các phương pháp đảm bảo tính nhất quán dữ liệu giữa các nhân. Đây là cơ sở lý thuyết quan trọng để triển khai các thiết kế hệ thống trên chip (SoC) có hiệu năng cao.

1 Introduction

1.1 Mục tiêu tìm hiểu

Tập trung vào các mục tiêu sau:

- Kiến trúc hệ thống: Phân tích cách tổ chức nhiều nhân xử lý trên cùng một đế chip (CMP - Chip Multiprocessor).
- Phân cấp bộ nhớ: Tìm hiểu cấu trúc bộ nhớ đệm (Cache) riêng biệt (L1) và dùng chung (L2/L3).
- Cơ chế liên kết (Interconnect): Nghiên cứu cách các nhân trao đổi dữ liệu thông qua Bus hoặc Network-on-Chip (NoC).

1.2 Các vấn đề kỹ thuật trọng tâm

1. Làm thế nào để duy trì tính nhất quán bộ nhớ (Cache Coherence) khi nhiều nhân cùng truy cập vào một vùng dữ liệu?
2. Sự khác biệt về hiệu năng giữa kiến trúc đơn nhân và đa nhân trong các bài toán tính toán song song.
3. Các thách thức về quản lý năng lượng và phân phối luồng (Thread scheduling) ở mức phần cứng.

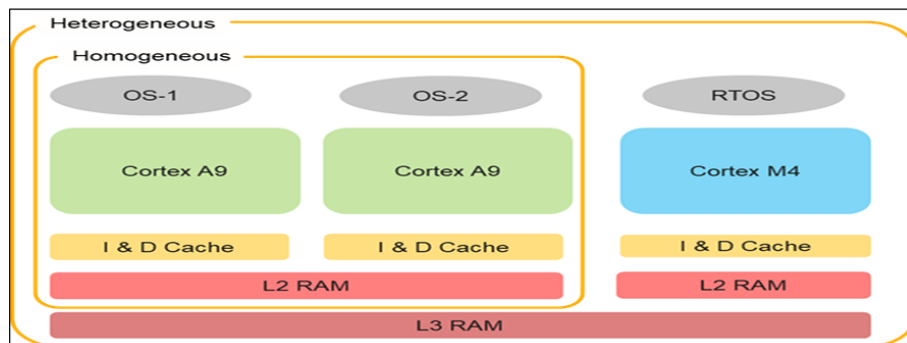
2 Kiến trúc Vi xử lý Đa nhân (Multicore Architecture)

2.1 Khái niệm và sự ra đời:

Kiến trúc vi xử lý đa nhân (Multicore) là việc tích hợp hai hay nhiều đơn vị xử lý độc lập (gọi là nhân hoặc core) vào cùng một mạch tích hợp (IC) hoặc trên cùng một đế silicon (die). Sự ra đời của kiến trúc này xuất phát từ việc thiết kế vi xử lý đơn nhân đã chạm đến giới hạn vật lý. Khi cố gắng tăng xung nhịp (clock rate) để tăng hiệu năng, lượng nhiệt tỏa ra và điện năng tiêu thụ tăng theo cấp số nhân (hiện tượng Bức tường nhiệt - Power Wall). Do đó, việc chuyển sang thiết kế nhiều nhân chạy ở xung nhịp thấp hơn trở thành giải pháp tối ưu để cải thiện hiệu suất mà vẫn đảm bảo hiệu năng trên mỗi watt điện.

2.2 Phân loại kiến trúc:

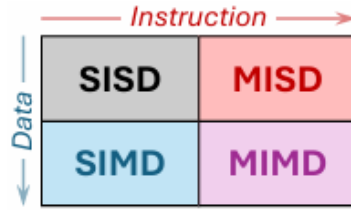
- **Kiến trúc đồng nhất (Homogeneous Multicore):** Tất cả các nhân trong hệ thống đều có cùng cấu trúc vi kiến trúc, tập lệnh và hiệu năng (Ví dụ: các dòng chip xử lý trung tâm đa lõi tiêu chuẩn). Giảm chi phí thiết kế, kiểm thử và sản xuất nhờ cơ chế tiling (thiết kế một nhân chuẩn rồi "dán" lặp lại nhiều lần trên chip). Thường thấy trong các CPU server hoặc máy tính bàn truyền thống nơi các tác vụ đòi hỏi sức mạnh đồng đều.
- **Kiến trúc không đồng nhất (Heterogeneous Multicore):** Tích hợp các nhân có vi kiến trúc và hiệu năng khác nhau để tối ưu hóa cho các tác vụ cụ thể. Điển hình là kiến trúc ARM big.LITTLE, kết hợp các nhân hiệu năng cao (ăn điện nhiều) với các nhân tiết kiệm năng lượng để cân bằng giữa hiệu suất và thời lượng pin.
 - **Ví dụ điển hình: ARM big.LITTLE:** Kết hợp nhân Cortex-A15 (mạnh) và Cortex-A7 (yếu).
 - * **Apple A17 Pro:** Có 2 nhân hiệu năng cao và 4 nhân tiết kiệm điện.
 - * **Intel 12th Gen (Alder Lake):** Chia thành P-cores và E-cores cho máy tính để bàn.



Hình 1: Homogeneous và Heterogeneous

2.2.1 Phân loại theo luồng lệnh và dữ liệu (Flynn's Taxonomy)

- Đa nhân được xếp vào nhóm MIMD (Multiple Instruction, Multiple Data) vì mỗi nhân xử lý các luồng lệnh và dữ liệu độc lập.

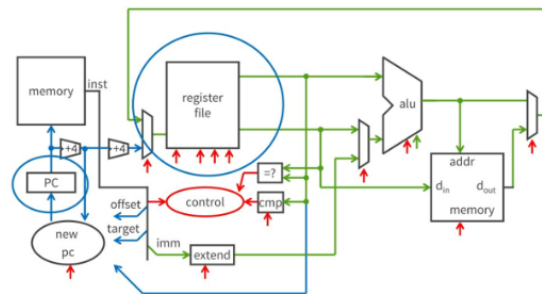


Hình 2: Sơ đồ phân loại của Flynn

- SIMD (Single Instruction, Multiple Data): Thường dùng cho các bộ xử lý đồ họa (GPU) hoặc các lệnh vector mở rộng.
 - SISD (Single Instruction, Single Data): Kiến trúc CPU đơn nhân truyền thống.
 - MISD (Multiple Instruction, Single Data): Ít phổ biến, thường dùng trong các hệ thống đòi hỏi tính dư thừa cao như điều khiển máy bay.
 - MIMD (Multiple Instruction, Multiple Data): Đây là nhóm mà các hệ thống đa nhân (Multicore) thuộc về. Mỗi nhân xử lý một luồng lệnh riêng biệt trên tập dữ liệu riêng của nó.
- Mỗi nhân có bộ điều khiển (control logic) riêng để thực hiện các luồng lệnh độc lập trên các tập dữ liệu khác nhau.

2.3 Cấu trúc bên trong một nhân (Core):

Một nhân cơ bản (như lõi RISC-V RV32I) sẽ bao gồm các khối Datapath và Control Unit hoạt động theo cơ chế Pipeline. Các khối chức năng chính yếu bao gồm: Bộ đếm chương trình (Program Counter), Bộ lấy lệnh (Instruction Fetch), Giải mã lệnh (Instruction Decode), Tập thanh ghi (Register File), Khối tính toán số học và logic (ALU), và giao tiếp với bộ nhớ đệm nội bộ (L1 Cache).



Hình 3: Cấu trúc 1 nhân cơ bản

3 Hệ thống Bộ nhớ và Kết nối (Memory & Interconnect)

3.1 Phân cấp bộ nhớ đệm (Cache Hierarchy):

Để giảm độ trễ truy cập bộ nhớ chính (DRAM), hệ thống đa nhân sử dụng kiến trúc bộ nhớ đệm nhiều cấp:

- **L1 Cache (Private):** Nằm ngay trong mỗi nhân, tốc độ cao nhất, thường chia làm L1 Instruction Cache (lưu lệnh) và L1 Data Cache (lưu dữ liệu).
- **L2 Cache:** Thường lớn hơn L1, có thể là private cho từng nhân hoặc dùng chung (shared) cho một cụm nhân.
- **L3 Cache (Shared):** Bộ nhớ đệm dùng chung cho toàn bộ các nhân trên chip, dung lượng lớn nhất và giúp đồng bộ dữ liệu giữa các nhân trước khi phải truy xuất ra DRAM.

Cách tổ chức quản lý thực tế:

- **Phân mảnh Cache (Cache Slicing):** Thay vì một khối Cache L3 khổng lồ (monolithic), các CPU hiện đại chia nhỏ L3 thành các "slice" (mảnh) để giảm nhiễu và tăng số lượng cổng truy cập đồng thời. Mỗi slice thường được quản lý bởi một bộ điều khiển.
- **Truy cập bộ nhớ không đồng nhất (NUCA - Non-Uniform Cache Access):** Giải thích rằng khi cache bị phân mảnh, thời gian để một nhân truy cập vào mảnh cache ở xa (remote slice) sẽ lâu hơn mảnh ở gần (local slice) do độ trễ đường truyền interconnect.
- **Thanh ghi trạng thái bỏ lỡ (MSHRs):** Đây là thành phần cực kỳ quan trọng giúp Cache tiếp tục xử lý các yêu cầu khác trong khi đang chờ dữ liệu từ DRAM nạp về (non-blocking cache). Nó hoạt động như một "bàn quản lý" các yêu cầu đang chờ dữ liệu từ RAM. Việc có nhiều MSHRs cho phép các nhân tiếp tục thực hiện các lệnh khác mà không bị "treo" khi chờ đợi bộ nhớ.

3.2 Tính nhất quán bộ nhớ đệm (Cache Coherence):

Khi nhiều nhân cùng lưu trữ một bản sao dữ liệu từ bộ nhớ chính vào L1 Cache của riêng nó, nếu một nhân thay đổi giá trị, các nhân khác phải được thông báo để tránh việc sử dụng dữ liệu cũ (stale data). Ở mức độ phần cứng, vấn đề này được giải quyết bằng các giao thức Cache Coherence, phổ biến nhất là giao thức Snooping (như MESI - Modified, Exclusive, Shared, Invalid) hoặc Directory-based để theo dõi trạng thái của từng block nhớ.

- **Coherence (Nhất quán bộ nhớ đệm):** Đảm bảo các nhân thấy giá trị mới nhất của một địa chỉ nhớ.
- **Consistency (Nhất quán bộ nhớ):** Quy định thứ tự thực hiện các lệnh đọc/ghi đối với nhiều địa chỉ nhớ khác nhau từ góc nhìn của lập trình viên.
- **Hiện tượng "Chia sẻ giả" (False Sharing):** Xảy ra khi hai nhân cùng truy cập vào các địa chỉ dữ liệu khác nhau nhưng vô tình nằm chung trên một "khối" bộ nhớ đệm (cache block). Hệ quả giao thức MESI sẽ hiểu nhầm là dữ liệu đang bị tranh chấp, khiến khối dữ liệu bị "nhảy qua nhảy lại" (ping-pong) giữa các nhân, làm sụt giảm hiệu năng nghiêm trọng mặc dù các nhân không dùng chung dữ liệu thực tế.

3.3 Cơ chế liên kết nội bộ (Interconnect):

Để các nhân và bộ nhớ giao tiếp được với nhau, hệ thống cần một mạng lưới kết nối:

- **Shared Bus:** Cấu trúc đơn giản, các nhân cùng kết nối vào một trục đường chung (như giao thức AXI Bus). Tuy nhiên, dễ nghẽn cổ chai khi số lượng nhân tăng lên.

- **Crossbar Switch:** Kết nối ma trận cho phép nhiều luồng giao tiếp diễn ra đồng thời, băng thông cao nhưng tốn diện tích phần cứng.
- **Network-on-Chip (NoC):** Sử dụng các Router và đường truyền (Link) để định tuyến gói tin (packet) giữa các nhân giống như một mạng máy tính thu nhỏ, đây là xu hướng cho các SoC có hàng chục nhân.
- **Ring Interconnect (Kết nối vòng):** Đây là cơ chế cực kỳ phổ biến trong các CPU đa nhân của Intel và các SoC hiện đại. Hệ thống này giúp cân bằng giữa độ phức tạp của Bus truyền thống và hiệu năng của NoC, đặc biệt tối ưu cho các hệ thống có số lượng nhân xử lý vừa phải (dưới 24 nhân).

3.4 Quản lý bộ nhớ chính (DRAM)

- **Xếp hàng địa chỉ (Address Interleaving):** Cách ánh xạ địa chỉ vào các Bank/Rank của DRAM để tối ưu hóa tính song song mức bộ nhớ (MLP).
- **Lập lịch bộ nhớ (Memory Scheduling):** Giới thiệu thuật toán FR-FCFS (First-Ready, First-Come-First-Serve), ưu tiên các yêu cầu truy cập vào cùng một hàng (row) đang mở trong DRAM để giảm độ trễ.
- **Giảm thiểu nhiễu (Interference Mitigation):** Đề cập đến kỹ thuật phân vùng Cache (Cache Partitioning) như Intel CAT để đảm bảo IP quan trọng không bị các ứng dụng khác chiếm dụng hết tài nguyên cache.

Trong DRAM, dữ liệu được tổ chức theo hàng (Row) và cột (Column). Để đọc một dữ liệu, phải thực hiện 2 bước tốn thời gian:

- **Mở ngăn kéo (Activate Row):** Kéo cả một hàng dữ liệu ra đặt lên bàn (Row Buffer)
- **Đóng ngăn kéo (Precharge):** Đẩy hàng đó về chỗ cũ trước khi mở hàng khác.

Thuật toán FR-FCFS hoạt động theo 2 quy tắc ưu tiên:

1. **Ưu tiên 1 - First-Ready (Sẵn sàng trước):** Bộ lập lịch sẽ ưu tiên phục vụ các yêu cầu truy cập có địa chỉ nằm trên cùng một hàng đang được mở sẵn trong Row Buffer. Điều này giúp loại bỏ hoàn toàn độ trễ của các lệnh Activate và Precharge.
2. **Ưu tiên 2 - First-Come, First-Serve (Đến trước phục vụ trước):** Trong trường hợp không có yêu cầu nào thỏa mãn điều kiện Row-Hit, thuật toán mới tiến hành chọn yêu cầu có thời gian chờ lâu nhất trong hàng đợi để xử lý.

Ưu điểm của thuật toán FR-FCFS:

- **Giảm độ trễ trung bình (AMAT):** Việc mở một hàng và đóng nó có thể tốn thời gian tương đương với chính thao tác đọc/ghi. Bằng cách ưu tiên "Row Hit" (truy cập vào hàng đang mở), thuật toán này giúp tăng tốc độ xử lý đáng kể.
- **Tối ưu hóa băng thông:** Giúp đường truyền dữ liệu (Data Bus) hoạt động liên tục hơn thay vì phải chờ đợi các lệnh Activate/Precharge.

Hạn chế của thuật toán FR-FCFS:

- **Hiện tượng "Luồng tham lam":** Nếu một nhân (ví dụ nhân đang chạy ứng dụng xử lý video) liên tục yêu cầu dữ liệu trên cùng một hàng đang mở, FR-FCFS sẽ ưu tiên nó mãi mãi.
- **Bỏ đói (Starvation):** Các nhân khác (ví dụ nhân đang chạy hệ điều hành) chỉ cần lấy một mẫu dữ liệu ở hàng khác sẽ phải xếp hàng chờ rất lâu.

4 Hiệu năng và Thách thức (Performance & Challenges)

4.1 Các mô hình đánh giá:

4.1.1 Định luật Amdahl (Fixed-size):

Định luật Amdahl chỉ ra giới hạn của việc tăng tốc độ tính toán khi sử dụng hệ thống đa nhân. Tốc độ cải thiện cực đại bị giới hạn bởi phần mã lệnh buộc phải thực thi tuần tự. Công thức:

$$S = \frac{1}{(1 - f) + \frac{f}{N}}$$

Trong đó: S là mức tăng tốc (Speedup), f là tỷ lệ phần trăm chương trình có thể chạy song song, và N là số lượng nhân xử lý.

4.1.2 Định luật Gustafson (Scaled-size):

Giả định rằng khi có nhiều tài nguyên hơn (nhiều nhân hơn), quy mô bài toán sẽ được mở rộng tương ứng. Công thức:

$$S = (1 - f) + f \times n$$

Trong đó: S là mức tăng tốc (Speedup), f là tỷ lệ phần trăm chương trình có thể chạy song song, và N là số lượng nhân xử lý.

4.2 Vấn đề thực thi song song:

Phần cứng đa nhân mạnh mẽ đòi hỏi phần mềm phải được viết dưới dạng đa luồng (Multithreading). Thách thức đặt ra là việc quản lý chia sẻ tài nguyên phần cứng, tránh các lỗi như Data Race (đua tranh dữ liệu) hay Deadlock (khóa cứng). Bộ định tuyến luồng (Thread Scheduler) của hệ điều hành phải liên tục phân phối luồng công việc hợp lý để không có nhân nào bị quá tải hay rảnh rỗi.

- **Tranh chấp tài nguyên dùng chung:** Ngoài lỗi phần mềm, phần cứng đa nhân đối mặt với hiện tượng nhiễu (interference) khi các nhân tranh giành bộ nhớ đệm dùng chung (LLC - Last Level Cache) và băng thông bộ nhớ chính.
- **Hiện tượng "Luồng tham lam":** Một nhân chạy ứng dụng tốn nhiều bộ nhớ có thể chiếm dụng toàn bộ LLC, gây ra việc liên tục đẩy (evict) dữ liệu của các nhân khác, làm sụt giảm hiệu năng hệ thống nghiêm trọng.
- **Tính công bằng (Fairness):** Cần có các bộ lập lịch bộ nhớ (Memory Scheduler) thông minh (như FR-FCFS cải tiến) để đảm bảo không có luồng nào bị "đói" tài nguyên (starvation).

4.3 Quản lý năng lượng và Tản nhiệt:

Với hàng tỷ bóng bán dẫn hoạt động, mật độ nhiệt trên chip đa nhân là một thách thức cực đại. Khái niệm "Dark Silicon" xuất hiện khi không thể bật tất cả các nhân cùng lúc ở xung nhịp tối đa do giới hạn tản nhiệt (TDP). Các kỹ thuật như DVFS (Dynamic Voltage and Frequency Scaling - Tự động thay đổi điện áp và xung nhịp) hoặc tắt hoàn toàn các nhân không sử dụng (Clock Gating / Power Gating) ở cấp độ RTL là bắt buộc để duy trì độ bền của chip.

4.4 Các chỉ số đo lường thực tế:

- **Weighted Speedup (WS):** Tổng hợp tốc độ của nhiều ứng dụng chạy đồng thời để đo thông lượng hệ thống.
- **Harmonic Speedup (HM):** Đo lường thời gian phản hồi trung bình của người dùng đối với tất cả ứng dụng trong hệ thống

5 Tài liệu tham khảo (References)

Tài liệu

- [1] Balaji Venu, "Multi-core processors - An overview,"Department of Electrical Engineering and Electronics, University of Liverpool, UK.
- [2] S.Borkar, "Thousand core chips - A technology perspective,"DAC'07: Proc. of the 44th annual Conf. on Design Automation.
- [3] Gordon Moore, "Moore's law: past, present and future,"Spectrum, IEEE.
- [4] <https://arxiv.org/pdf/2408.12999v1>
- [5] <https://ijisrt.com/assets/upload/files/IJISRT24MAY458.pdf>